

私が確かめたかったこと

無料・静的サイトだけで「本当に役立つ避難案内」は自分で作れるのか

開発メンバー勉強会 / 課題1：アプリケーションシステム作成
Kyoko Takazawa

この発表は「問い」から始まります

「すごいものを作りました」報告ではありません。
個人的に確かめたいことがあって、それを軸に作りました。

出発点になった2つの実感

① 既存のハザードマップは情報が多く、いざという時
「で、今どこへ逃げれば？」に即答してくれない

② 災害アプリなのにサーバ頼み...でも
被災時は通信が落ちる
。お金もかけたくない。

問い：お金もサーバもかけず、無料・静的だけで、
「今どこへ逃げれば安全か」を即答する"役立つ"避難案内は作れるのか？

作ったもの（問いを試すための題材）

災害時避難シミュレーター

災害種別に応じて、避難所とルートを提案する
Web アプリ

- 地震／津波／台風／洪水／土砂で提案が変わる
- 危険区域（浸水域）を避けるルート
- オフライン対応・OIDC認証・HTTPS

 <https://kyokoron.github.io/team-challenge/>
(完全無料・GitHub Pages)

以降、このアプリを通して「無料・静的で役立つものは作れたか」を検証していきます。

検証 ①

「災害時に本当に役立つか」を確かめる

「役立つ」を作ろうとして、逆に怖くなった

動くものはすぐできた。でも触ってみて危うさに気づいた。
避難案内は、間違えると命に関わる。

内陸で「津波」を選ぶと海へ8km誘導

指定避難所が沿岸に偏在 → 逆方向は致命的

架空のサンプルデータが出得た

取得失敗時に"偽の避難所"を表示する余地

"最寄り"が最寄りでない

広域データで距離が正規化され順位が崩れる

直線距離を「徒歩〇分」表示

川や線路で実際は遠い = 過小表示で油断を生む

「役立つ」の前に、まず「間違えない」だった。

気づき：「安全側に倒す」まで作って初めて"役立つ"

- 対象外の地点は遠くへ誘導せず「垂直避難を」と警告
- 偽データを完全排除（取れなければ正直に止める）
- 現在地が浸水域内なら最優先で警告（右図）
- 直線距離は**「実際の道のりは異なります」と正直に**表示
- できないこと（危険の完全自動回避は不可等）も明示

検証①の答え：役立つものは作れる。ただし"役立つ"の中身は「間違えない設計」だった。

難）へ移動してください。

荒木田ふれあい館

第1候補

- ・ 直線 約2090m（実際の道のりは異なります）
- ✓ 洪水の指定避難場所

実ルート：徒歩約5分（0.40km）／ 浸水想定区域を回避した経路

第七峡田小学校

第2候補

- ・ 直線 約2175m（実際の道のりは異なります）
- ✓ 洪水の指定避難場所

原中学校

第3候補

- ・ 直線 約2205m（実際の道のりは異なります）
- ✓ 洪水の指定避難場所

第五峡田小学校

第4候補

- ・ 直線 約2374m（実際の道のりは異なります）
- ✓ 洪水の指定避難場所

検証 ②

「無料・静的の限界」を確かめる

サーバなしは"制約"のはずが、思想と噛み合った

- 災害アプリの本質 = 通信が落ちる瞬間に使えること
- サーバDBは「その瞬間に頼れない」 → 静的配信 + 端末保存が理にかなう
- オフライン(PWA) : 一度見た地域は IndexedDB に保存、圏外でも検索可
- OIDC認証(Auth0) を バックエンドなし (PKCE) で実現
- 浸水域を避けるルート (ORSの `avoid_polygons`)
- すべて **GitHub Pages** (静的・無料・HTTPS)

「無料・静的」は妥協ではなく、この題材にはむしろ正解だった。

事前保存)

済み (オフライン可)

しました (次回オフラ

もちろん限界もあった（正直に）

できた

避難所ランキング／浸水回避ルート／オフライン検索／認証／HTTPS を、サーバ0円で

できなかった・限界

リアルタイム更新・混雑や通行止め・全国全ハザードの即時反映は、静的だけでは無理

学び：本当に全国運用するなら「平常時にサーバで整えて、端末に配って被災時に使う」

ハイブリッド

が現実解。無料・静的は

"個人が価値を検証する"段階には十分すぎる

。

検証②の答え：無料・静的で"実用的な核"は作れる。限界の線も、作ってみて初めてはっきり引けた。

両方の土台：オープンデータのリアル

- 避難所・標高・ハザードは 国のオープンデータで揃う（無料・商用可も）
- が、実際に使うと...
 - どれが正解のデータか 分かりにくい（避難所は複数種別）
 - 洪水浸水想定(A31)は 河川単位で「東京の1ファイル」が無い
 - **Shapefile** が多く、GeoJSON化・座標変換・簡略化の前処理が要る
 - ルートAPI(ORS)は 回避ポリゴンに上限があり工夫が必要

「データはある」と「すぐ使える」は別物。
整形・前処理が実装と同じくらい重い
と痛感した。

補足：手段として使った「AI開発」の所感

AIで作ってみて — 速いが、丸投げは危ない

この検証、**Claude Code** と対話しながら進めた。課題2・3(AI開発)にも通じる所感：

AIが得意	人間の判断が必要	危なかった点
叩き台の高速生成	何を・なぜ作るか	"それっぽく動く"が正しくない
データ変換・前処理	安全・倫理の線引き	偽データfallbackを平然と残す
CSS/UI調整の反復	優先順位づけ	誤ランキングに気づかない
定型実装・リファクタ	「この挙動おかしい」の指摘	レビューを怠ると事故

前述の"危うさ"は全て、私が触って指摘して直した。速度はAI、判断とレビューは人。

AI時代、プロダクトの「価値の中心」が動く

従来



AI時代



コードを書くことよりも、「何を解くべきか」の方が重要になる。

今回、私の「価値の中心」もそこだった

図の3つは、まさに今回の私が握っていた部分だった。

① ドメイン理解

防災の安全要件。「津波で海へ誘導」の危うさに気づけたのは、コードではなく人の理解

② 良いデータ

国のオープンデータを"使える形"に整える前処理が、品質を大きく左右した

③ 何を解くべきか

「動く」より「間違えない」。問いを立て直し、レビューし続けた

= 実装の速さはAIに任せ

私はこの3つに集中していた（と、後から気づいた）

余談：この資料も Markdown (Marp) で作った

- スライドを **文章 (テキスト)** で書く ツール。図を並べる代わりに、構造を文字で書く
- なぜ AI 時代に相性がいい？
 - **AIが読める・書ける・直せる** — 「ここ直して」で即反映 (この資料が現にそう)
 - **Git で差分管理** できる = スライドをコードと同じように扱える
 - **中身とデザインが分離** (テーマは固定、中身だけ高速に直せる)
- パワポのように1枚ずつ手で整えるより、**中身の議論に集中**できた

所感：AI時代は「テキストで伝える → AIが形にする」が中心。

スライドもテキストで持てるMarpは、その波に乗りやすい

。ただし作り込んだビジュアル勝負なら、Canvaや従来ツールが今も有利。

苦労・失敗したこと

- 動いて見えて、裏で **"偽データ・誤順位"** が潜んでいた
- データ入手・変換で何度もつまずいた（河川単位・Shapefile・API制限）
- 認証やオフラインは **一度で動かず**、原因切り分けを繰り返した
- 「見た目が微妙」を直すのが一番難しい（**"良い"の言語化**が難しく作り直し多数）

失敗の多くは「動かして初めて分かる」もの。

触る→気づく→直す

の反復が、結局いちばん効いた。

まとめ：問いへの答え

Q. 無料・静的だけで「役立つ避難案内」は作れるか？

- A. 作れる。ただし "役立つ" の正体は機能の多さではなく 「間違えない設計」 だった
- 無料・静的 は妥協ではなく、被災時に使うという 思想と噛み合えば強み。限界の線も自分で引けた
- オープンデータは宝の山だが、前処理という現実がある
- AIで速度は劇的に上がる。でも "正しさ・安全"を問い続けるのは人の仕事

作って一番の収穫は、成果物そのものより「どこを自分が握るべきか」が分かったこと。

付録：成果物の概要

機能・技術・データ（付録）

主な機能

- 災害種別に応じた避難所ランキング（理由つき）
- 浸水域を避ける避難ルート／垂直避難の警告
- 現在地の浸水域内 警告
- オフライン（PWA）／OIDC認証 + HTTPS

 <https://kyokoron.github.io/team-challenge/>

技術・データ

- Vanilla JS / MapLibre GL JS / Auth0(OIDC)
- Service Worker + IndexedDB
- 国土地理院（地図・避難所・標高）
- 国土数値情報（洪水浸水想定 A31）
- OpenRouteService（回避ルート）